

Bash Star Pattern Generator

Comprehensive step-by-step line explanation, mathematical analysis, and execution traces for

`drawStar.sh`

1. Script Overview & Architecture

The provided script is an interactive pattern printing utility written in **Bash shell scripting**. It takes two command-line arguments (`size` and `type`) and renders formatted geometric star (`*`) shapes in the terminal.

Complete Source Code (`drawStar.sh`)

```
#!/bin/bash

# Check arguments
if [ $# -ne 2 ]; then
    echo "Usage: $0 <size> <type>"
    exit 1
fi

size=$1
type=$2

case $type in

t1)
# Right triangle (right aligned)
for ((i=1;i<=size;i++))
do
    for ((j=size-i;j>=1;j--))
    do
        printf " "
    done
    for ((k=1;k<=i;k++))
    do
        printf "*"
    done
    echo
done
;;

t2)
# Left aligned triangle
for ((i=1;i<=size;i++))
do
    for ((j=1;j<=i;j++))
    do
        printf "*"
    done
    echo
done
;;

t3)
# Pyramid
for ((i=1;i<=size;i++))
do
    for ((j=size-i;j>=1;j--))
    do
        printf " "
    done

    for ((k=1;k<=2*i-1;k++))
    do
        printf "*"
    done
    echo
done
;;

t4)
# Inverted left triangle
for ((i=size;i>=1;i--))
do
```

```

        for ((j=1;j<=i;j++))
        do
            printf "*"
        done
        echo
    done
;;

t5)
# Inverted right triangle
for ((i=size;i>=1;i--))
do
    for ((j=1;j<=size-i;j++))
    do
        printf " "
    done

    for ((k=1;k<=i;k++))
    do
        printf "*"
    done
    echo
done
;;

t6)
# Inverted pyramid
for ((i=size;i>=1;i--))
do
    for ((j=1;j<=size-i;j++))
    do
        printf " "
    done

    for ((k=1;k<=2*i-1;k++))
    do
        printf "*"
    done
    echo
done
;;

t7)
# Diamond

# Upper Pyramid
for ((i=1;i<=size;i++))
do
    for ((j=size-i;j>=1;j--))
    do
        printf " "
    done

    for ((k=1;k<=2*i-1;k++))
    do
        printf "*"
    done
    echo
done

# Lower Pyramid
for ((i=size-1;i>=1;i--))
do

```

```

        for ((j=1;j<=size-i;j++))
        do
            printf " "
        done

        for ((k=1;k<=2*i-1;k++))
        do
            printf "*"
        done
        echo
    done
;;

*)
echo "Invalid Type"
;;

esac

```

2. Fundamental Components & Initialization Block

1. Shebang Line (`#!/bin/bash`)

Informs the operating system kernel to interpret and execute this file using the Bash binary located at `/bin/bash`.

2. Command-Line Argument Verification

```

if [ $# -ne 2 ]; then
    echo "Usage: $0 <size> <type>"
    exit 1
fi

```

- `$#` : A special Bash variable holding the exact count of positional parameters passed.
- `-ne 2` : Evaluates to true if the argument count is **Not Equal** to 2.
- `$0` : Expands to the filename/path used to run the script (e.g., `./drawStar.sh`).
- `exit 1` : Terminates script execution immediately with an error exit code (1) to prevent undefined operations.

3. Variable Assignment

```

size=$1
type=$2

```

- `size=$1` : Stores the first argument (e.g., `5`), representing pattern height/rows.
- `type=$2` : Stores the second argument (e.g., `t1` through `t7`), selecting the target shape.

4. Pattern Routing via `case...esac`

The `case $type in ... esac` block matches the value of `$2` against valid choices (`t1` to `t7`). If an unmatched string is supplied, the wildcard branch `*)` executes, printing `"Invalid Type"`.

3. Loop Index Variables & Math Rules

Text terminals render output left-to-right, line-by-line. To build 2D shapes, three nested loop variables are employed:

- `i` (Row Counter): Outer loop variable tracking the vertical line number ($1 \leq i \leq \text{size}$).
- `j` (Space Counter): First inner loop tracking left padding spaces required for horizontal alignment.
- `k` (Star Counter): Second inner loop emitting star characters (`*`) for visible geometry.

4. Step-by-Step Breakdown of All 7 Pattern Options

Pattern t1 — Right-Aligned Right Triangle

Logic: On row `i`, print `size - i` spaces followed by `i` stars.

```
for ((i=1;i<=size;i++)); do
  for ((j=size-i;j>=1;j--)); do printf " "; done
  for ((k=1;k<=i;k++)); do printf "*"; done
  echo
done
```

Trace Table (size = 4):

Row (i)	Space Count (size - i)	Star Count (i)	Visual Output
1	4 - 1 = 3 spaces	1 star	*
2	4 - 2 = 2 spaces	2 stars	* *
3	4 - 3 = 1 space	3 stars	* * *
4	4 - 4 = 0 spaces	4 stars	* * * *

Pattern t2 — Left-Aligned Right Triangle

Logic: On row *i*, no leading spaces are required. Directly print *i* stars.

```
for ((i=1;i<=size;i++)); do
  for ((j=1;j<=i;j++)); do printf "*"; done
  echo
done
```

Trace Table (size = 4):

Row (i)	Spaces	Star Count (i)	Visual Output
1	0	1 star	*
2	0	2 stars	**
3	0	3 stars	***
4	0	4 stars	****

Pattern t3 — Centered Pyramid

Logic: Centering requires *size - i* spaces on the left and an odd number of stars, *2i - 1*, on each row *i*.

```
for ((i=1;i<=size;i++)); do
  for ((j=size-i;j>=1;j--)); do printf " "; done
  for ((k=1;k<=2*i-1;k++)); do printf "*"; done
  echo
done
```

Trace Table (size = 4):

Row (i)	Space Count (size - i)	Star Formula (2i - 1)	Visual Output
1	3 spaces	2(1) - 1 = 1 star	*
2	2 spaces	2(2) - 1 = 3 stars	***
3	1 space	2(3) - 1 = 5 stars	*****
4	0 spaces	2(4) - 1 = 7 stars	*****

Pattern t4 — Inverted Left Triangle

Logic: Loop *i* decrements from *size* down to 1. On each row, print *i* stars directly.

```
for ((i=size;i>=1;i--)); do
  for ((j=1;j<=i;j++)); do printf "*"; done
  echo
done
```

Trace Table (size = 4):

Row (i)	Spaces	Star Count (i)	Visual Output
4	0	4 stars	****
3	0	3 stars	***
2	0	2 stars	**
1	0	1 star	*

Pattern t5 — Inverted Right Triangle

Logic: Loop *i* decrements from *size* to 1. Print *size - i* spaces followed by *i* stars.

```
for ((i=size;i>=1;i--)); do
  for ((j=1;j<=size-i;j++)); do printf " "; done
  for ((k=1;k<=i;k++)); do printf "*"; done
  echo
done
```

Trace Table (size = 4):

Row (i)	Space Count (size - i)	Star Count (i)	Visual Output
4	4 - 4 = 0 spaces	4 stars	****
3	4 - 3 = 1 space	3 stars	***
2	4 - 2 = 2 spaces	2 stars	**
1	4 - 1 = 3 spaces	1 star	*

Pattern t6 — Inverted Pyramid

Logic: Decrement *i* from *size* to 1. On each row, print *size - i* spaces and *2i - 1* stars.

```
for ((i=size;i>=1;i--)); do
  for ((j=1;j<=size-i;j++)); do printf " "; done
  for ((k=1;k<=2*i-1;k++)); do printf "*"; done
  echo
done
```

Trace Table (size = 4):

Row (i)	Space Count (size - i)	Star Formula (2i - 1)	Visual Output
4	0 spaces	2(4) - 1 = 7 stars	*****
3	1 space	2(3) - 1 = 5 stars	*****
2	2 spaces	2(2) - 1 = 3 stars	***
1	3 spaces	2(1) - 1 = 1 star	*

Pattern t7 — Diamond

Logic: Combines an upper pyramid of height *size* (rows *1 \dots size*) and a lower pyramid of height *size - 1* (rows *size - 1 \dots 1*) so that the central row is not repeated.

Visual Structure (size = 4):

Section	Row (i)	Spaces (size - i)	Stars (2i - 1)	Output
Upper Pyramid	1	3	1	*
Upper Pyramid	2	2	3	***
Upper Pyramid	3	1	5	*****
Upper Pyramid	4 (Peak)	0	7	*****
Lower Pyramid	3	1	5	*****
Lower Pyramid	2	2	3	***
Lower Pyramid	1	3	1	*

5. Execution Permissions & Terminal Troubleshooting

1. Permission Denied Error

When running `./drawStar.sh 5 t1`, if the error `bash: ./drawStar.sh: Permission denied` appears, the file lacks execute permission flags. Fix by executing:

```
chmod +x drawStar.sh
```

2. Sudo Command Not Found Error

Running `sudo ./drawStar.sh` on a non-executable file returns

`sudo: ./drawStar.sh: command not found` because Linux security policies prevent `sudo` from invoking non-executable binaries.

3. Execution Without Changing Permissions

You can bypass the permission requirement by passing the script file directly into the Bash interpreter:

```
bash drawStar.sh 5 t1
```